

プログラム言語論

亀山幸義

筑波大学 情報科学類

No. 2

目次

- 1 プログラミング言語の種類
- 2 関数型プログラミング言語
- 3 OCaml プログラミング入門
- 4 OCaml の利用
- 5 演習

プログラム言語の種類

計算モデルに関する分類

- ▶ 手続き型言語 (Procedural PL): C 言語など非常に多数
- ▶ 宣言型言語 (Declarative PL)
 - ▶ 関数型言語: Lisp, Scheme, OCaml, Haskell など
 - ▶ 論理型言語: Prolog など

そのほかの分類としては、たとえば、関数型、論理型、オブジェクト指向型などの「パラダイム」に関する分類など。

目次

- ① プログラミング言語の種類
- ② 関数型プログラミング言語
- ③ OCaml プログラミング入門
- ④ OCaml の利用
- ⑤ 演習

関数型プログラミング言語

関数 (正確には、部分関数) の概念を中心に置いたプログラミングを行うための言語である。

- ▶ 手続き型言語における計算 == 「状態を更新して、次の状態にする手続き」
- ▶ 関数型言語における計算 == 「引数から返り値を計算する手続き」

C 言語の文 (statement): $x = x+3$; という文は、 $(x=n)$ という状態を $(x=n+3)$ という状態に更新する手続きをあらわす。

ML 言語の式 (expression): $\text{fun } x \rightarrow x*3$ という式は、整数 n をもらって $n+3$ を返す関数をあらわす。

代表的な関数型プログラム言語

- ▶ Lisp (Common Lisp など多数の種類あり)
- ▶ Scheme: Lisp の一種と見なすことが多い
- ▶ ML 系言語 (SML, OCaml, F# など)
 - ▶ SML... 正式には Standard ML (最初の ML 言語)
 - ▶ OCaml ... もともと Objective Caml だった。(Caml 言語にオブジェクト指向機能を追加した。)
- ▶ Haskell: 「純粋」な関数型言語

なぜ、関数型プログラム言語を学習するのか

仮定: この授業を受けている人は、「最初のプログラミング言語」として、Python, C, Java 等の言語を習っているはず。→ 手続き型言語の基礎はわかっているだろう。

なぜ関数型言語に基づいてプログラミング言語を考えるのか？

- ▶ 手続き型言語やオブジェクト指向言語より、関数型言語の方が**簡単**だから。

(関数型言語の型システムを理解したあと、オブジェクト指向言語などの、より複雑な型システムを理解することはできるが、逆はやりにくい。)

- ▶ 型理論の発展の歴史は、関数型言語の発展の歴史でもあるから。

参考: この授業では、最後の方で、Java (オブジェクト指向言語) の型システムについて学習する。

目次

- ① プログラミング言語の種類
- ② 関数型プログラミング言語
- ③ OCaml プログラミング入門
- ④ OCaml の利用
- ⑤ 演習

OCaml プログラミング (1)

いろいろな関数の定義 (キーワード `let` で始める)

```
let f1 x = x*x
let f2 x = (f1 (x+1) + f1 (x-1)) / 2
let f3 x y = f1 (x + (f2 y))
let f4 x = if x>10 then 10 else x
```

関数の利用

```
let ex1 = f3 5 10
(* ==> 11236 *)
let ex2 = (f4 5) + (f4 15)
(* ==> 15 *)
```

OCaml プログラミング (2)

再帰関数の定義 (キーワード `let rec` で始める)

```
let f5 x = if x=0 then 1 else f5 (x-1)+2
(* ==> エラー *)
let rec f5 x = if x=0 then 1 else f5 (x-1)+2
let ex3 = f5 10
(* ==> 21 *)
```

最大公約数を計算する関数

```
let rec gcd x y =  
  if x=0 then y  
  else if y=0 then x  
  else if x>y then gcd (x-y) y  
  else gcd (y-x) x
```

```
let ex4 = gcd 20 3  
(* ==> 1 *)  
let ex5 = gcd 20 4  
(* ==> 4 *)
```

OCaml プログラミング (4)

Fibonacci 数を計算する関数

```
let rec fib n =  
  if n <= 1 then 1  
  else (fib(n-2)) + (fib(n-1))  
let ex6 = fib 10  
(* ==> 89 *)  
let ex7 = fib 40  
(* とても時間がかかる *)
```

「組 (tuple)」のデータ構造を利用

```
let rec fib2 n =  
  if n = 0 then (1, 1)  
  else let (a, b) = fib2(n-1) in  
    (b, a+b)  
let ex8 = fib2 40  
(* ==> (165580141, 267914296) *)
```

OCaml プログラミング (5)

再帰の利用

```
(* 階乗の計算 *)  
let rec factorial n =  
    if n=0 then 1  
    else n * (factorial(n-1))  
let ex = factorial 10  
(* ==> 3628800 *)  
  
(* べき乗の計算 *)  
let rec power x n =  
    if n=0 then 1  
    else x * (power x (n-1))  
let ex = power 2 10  
(* ==> 1024 *)
```

OCaml プログラミング (番外編 1)

再帰でなく繰返しを使うことも可能 (この授業では推奨しない)

```
(* べき乗の計算 *)  
let power2 x n =  
  let k    = ref n in  
  let res = ref 1 in  
  while (!k > 0) do  
    res := !res * x;  
    k   := !k - 1  
  done;  
  !res    (* この値を返す *)  
let ex = power2 2 10  
(* ==> 1024 *)
```

OCaml プログラミング (番外編 2)

再帰ではあるが繰返しのように関数を呼び出す

```
(* べき乗の計算 *)  
let rec power3 x (n, res) =  
  if n=0 then res  
  else power3 x (n-1, res*x)  
let ex = power3 2 (10,1)  
(* ==> 1024 *)
```

power3 の第 2 引数が、
(10,1)=>(9,2)=>(8,4)=>(7,8)=>...=>(0,1024) と変化し、繰返し処理と実質的に同じ。

OCaml プログラミング (6)

Collatz 予想 (未解決問題):

```
let rec collatz n =  
  if n=1 then 1  
  else if n mod 2 = 0 then collatz(n/2)  
  else collatz(n*3+1)  
let ex = collatz 100  
(* ==> 1 *)
```

OCaml プログラミング (7)

関数もデータとなる (高階関数)

```
let compose f g x = f (g x)
let add1 x = x+1
let multiply3 x = x*3
let ex1 = compose add1 add1 10
(* ==> 12 *)
let ex2 = compose add1 multiply3 10
(* ==> 31 *)
```

関数を n 回呼び出す (高階関数)

```
let rec iterate f x n =
  if n=0 then x
  else iterate f (f x) (n-1)
let ex3 = iterate add1 10 5
(* ==> 15 *)
let ex3 = iterate multiply3 10 5
(* ==> 2430 *)
```

型付けについて少しだけ

OCaml は「静的」に「型付け」する言語である。

- ▶ 静的 (static): 実行時 (動的) より前、通常は「コンパイル時」のこと。
- ▶ 型付け (typing): 式やプログラムに、「型」を割り当てること。

これまでの関数の型:

```
f1, f2, f3, f4, f5      : int → int
fib, fib2, collatz      : int → int
factorial               : int → int
gcd, power              : int → int → int
```

高階関数 (compose と iterate) の型は難しいので後述。

まとめ

- ▶ 関数型プログラミング言語は、「引数から返り値への (部分) 関数」を組み合わせる形でプログラムを作成する。
- ▶ 関数の中には、「他の関数をデータとして受けとる」関数もあり、高階関数 (higher-order function) と呼ばれる。
- ▶ OCaml 言語は関数型プログラミング言語の 1 つであり、静的に型付けされる。

目次

- ① プログラミング言語の種類
- ② 関数型プログラミング言語
- ③ OCaml プログラミング入門
- ④ OCaml の利用
- ⑤ 演習

OCaml の利用法あれこれ

OCaml が動くコンピュータ

- ▶ 情報科学類 計算機室の Linux マシンで ocaml コマンド
 - ▶ 3C113 計算機室に行って直接使う。
 - ▶ 自分の PC から `ssh user@azalea01.coins.tsukuba.ac.jp` でログイン
 - ▶ (01 の部分は、30 までの異なる数にすると負荷分散できる)
- ▶ 自分の PC に OCaml を載せて使う
 - ▶ Linux: Debian 系なら `sudo apt-get install ocaml`
 - ▶ MacOS: `brew install ocaml`
 - ▶ Linux, MacOS でのより良い方法: 最初に opam を載せ、opam を使って OCaml を載せる。
 - ▶ Windows: WSL あるいは opam で可能
- ▶ ウェブサービスを使う
 - ▶ OCaml Playground: <https://ocaml.org/play>
 - ▶ Try OCaml: <https://try.ocamlpro.com>

OCaml のファイルの作成

Emacs エディタを使う場合

- ▶ tuareg をいれると、使い勝手が格段に向上する。
- ▶ `cat ~/kam/ocaml/tuareg.el >> ~/.emacs` とするとよい。

vscode でエディタを使う場合

- ▶ 拡張機能には例えば OCaml Platform がある
<https://github.com/ocaml-labs/vscode-ocaml-platform>
- ▶ 言語サーバのインストールなど依存解決が必要であり、詳しくは上記公式ページの Getting started の項を読むこと。
- ▶ OCaml ごと opam を使って実行環境を整える方が簡単かも。

OCaml 起動テスト

対話的に使えるかのテスト:

```
$ ocaml
      OCaml version 4.14.1
```

```
# 1 + 2 ;;
- : int = 3
#
```

(対話モードを終了したいときは、`ctl-d` などを押す)。

OCaml ソースコードがはいったファイル `test1.ml` を作成:

```
let f x = x + 1 ;;
```

上記の `test1.ml` を対話的に使う:

```
# #use "test1.ml";;
val f : int -> int = <fun>
# f 3 ;;
- : int = 4
```

OCaml ファイルのコンパイル

コンパイルして高速化したい場合、まずソースファイル test2.ml:

```
let f x = x + 1;;  
let res = print_endline (string_of_int (f 5)) ;;
```

上記をコンパイルして実行 (OCaml の外でやる):

```
$ ocamlc test2.ml      (a.out ファイルができる)  
$ ./a.out              (実行する)  
6                      (print されたものが表示される)
```

上記をネイティブコンパイルして実行 (OCaml の外でやる):

```
$ ocamlpt test2.ml     (a.out ファイルができる)  
$ ./a.out              (実行する)  
6                      (print されたものが表示される)
```

インターネット上の記事 (非常に多数あり、良質な解説も多い):

日本語の本:

- ▶ 浅井健一, 「プログラミングの基礎」, サイエンス社, 2007.
- ▶ 五十嵐淳, 「プログラミング in OCaml」, 技術評論社, 2007.

最新の情報:

- ▶ 日本の OCaml ユーザ: <https://ocaml.jp>
- ▶ OCaml の本家: <https://ocaml.org>

目次

- 1 プログラミング言語の種類
- 2 関数型プログラミング言語
- 3 OCaml プログラミング入門
- 4 OCaml の利用
- 5 演習

第1回演習問題

次ページ以降の関数を実装せよ。

- ▶ 1つ以上実装すること。(多い方が評価がよくなる。)
- ▶ 関数の引数および返り値はすべて `int` 型 (整数型) とする。また、今回の演習では、途中段階であっても `float` 型 (浮動小数点数型) は極力使わないこと。(たとえば、「べき乗」を `float` 型で計算する関数は OCaml のライブラリにあるが、そういうものはできるかぎり使わないこと。)
- ▶ 補助関数を使ってよい。(補助関数を使わないと解けない問題の方が多い。)
- ▶ テストを十分したコードを提出すること。

- ▶ 関数 f_0 :
引数 n と m (両方とも int 型) に対して、「2026 年 n 月 m 日」が存在するなら、1 を返し、存在しないなら、0 を返す。例: $f_0 6 31 = 0$, $f_0 7 31 = 1$, $f_0 (-5) 0 = 0$
- ▶ 関数 f_1 :
引数 n に対して、 n が正の整数なら、 n の正の約数の総和を返す。
例: $f_1 24 = 1 + 2 + 3 + 4 + 6 + 8 + 12 + 24 = 60$
- ▶ 関数 f_2 :
引数 n に対して、 n が正の整数なら、 n と互いに素な正整数の個数を返す。例: $f_2 24 = \#\{5, 7, 11, 13, 17, 19, 23\} = 7$

- ▶ 関数 $f3$: 引数 n が正の整数で、引数 k が非負の整数ならば、 n を二進数で表現したとき、下から $k + 1$ 桁目の数字 (0 か 1) を返す。たとえば、 $k = 0$ のとき最下位の桁の数字、 $k = 1$ は下から 2 桁目の数字を返す。

例: $f3\ 11\ 0 = 1$, $f3\ 11\ 1 = 1$, $f3\ 11\ 2 = 0$, $f3\ 11\ 3 = 1$,
 $f3\ 11\ 4 = 0$

(ヒント: OCaml の整数に対する割り算は、余りを切り捨てた答えを返す。整数上の割り算の余りを返す関数として `mod` がある。たとえば、 $10\ mod\ 3 = 1$)

演習問題の続き

- ▶ 関数 $f4$: 引数 n, m, k に対して、 n, m, k が正の整数であれば、 n と m と k の最小公倍数を返す。例: $f4\ 9\ 2\ 6 = 18$, $f4\ 10\ 14\ 6 = 210$

第1回演習問題(追加)のヒント

- ▶ 「引数 n が正のとき,,,,」という指定の場合, 引数 n が正でないときは, どんな値を返してもよいし, 無限ループでもよい.
- ▶ 条件式を書きたい時は `if e1 then e2` とせず, `if e1 then e2 else e3` のように必ず `else` をつけること. (`else` なしにすると, 型合わせが難しい.)
- ▶ 関数に引数を渡すとき, $f(x)$ でなく $f\ x$ と書いてもよい(後者が普通).
- ▶ マイナスの数は括弧をつけること. たとえば, $f\ -1$ というのは, $f\ -\ 1$ という意味になってしまうので, $f\ (-1)$ のように書くこと.
- ▶ 1つの再帰関数で定義できないものは, 補助関数を使ってもよい.

第1回演習問題(追加)のヒント

- ▶ この授業の中心テーマである「型の整合性」について、OCaml 言語は非常に厳格であり、`int` と `float` も異なる型である。つまり、`3.14 + 5` という式は型エラーになる。
- ▶ そこで、`float 30` とすると、整数型の `30` が浮動小数点型の `30.0` に変換される。また、`3.14 +. 5.1` のように `+.` という演算を使うと、浮動小数点型の加算となる。
- ▶ 今回の演習では、すべての数値データは整数型だけで表現でき、再帰関数をうまく使えば全ての処理を記述できるはずである。